Giải mã suy đoán exact trên mô hình 4-bit NF4 không dùng thêm bộ nhớ GPU: báo cáo kỹ thuật

Nguyễn Quốc Hiếu

Tháng 11 năm 2026

Bản nháp; toàn bộ kết quả là bằng chứng thực nghiệm hữu hạn trên một GPU, chưa phải chứng minh.

Tóm tắt nội dung

Giải mã suy đoán (speculative decoding) thường được coi là không làm đổi đầu ra, nhưng điều đó chỉ đúng khi bước kiểm tra theo khối và bước giải mã từng token cho *đúng cùng* giá trị số học. Với mô hình lượng tử hóa NF4 của bitsandbytes, hai đường này dùng kernel, kích thước tile và thứ tự rút gọn khác nhau, nên khối kiểm tra làm lệch quỹ đạo token so với greedy tuần tự. Báo cáo này trình bày *B2*, một vòng giải mã dùng tra cứu n-gram trong prompt (PLD, không cần mô hình draft) cho Qwen2.5-3B-Instruct NF4 và tái tạo *đúng từng token* cũng như bộ nhớ KV của greedy bitsandbytes gốc. B2 gồm các phép attention, chuẩn hóa và LM head gọi từng hàng với đúng hình dạng của bước một-token (chính các toán tử gây lệch khi kiểm tra theo khối, theo phép đo toán tử của chúng tôi), quay lui KV bằng cắt độ dài logic, quay về bước $M{=}1$ gốc khi không có đề xuất, chiếu LM head chỉ tới hàng bị loại đầu tiên, và một kernel Triton $M{=}3$ tùy chọn chỉ để tăng tốc thêm khoảng 4%. Trên hai cohort held-out GSM8K dùng đúng một lần (mỗi cohort 50 prompt), B2 khớp bitsandbytes ở $50/50$ prompt trong cả hai, nhanh hơn bitsandbytes eager khoảng $1{,}12$–$1{,}15\times$ (khoảng tin cậy 95% hẹp) và dùng thêm $0{,}07$ MiB bộ nhớ đỉnh. Chúng tôi cũng báo cáo các kết quả bất lợi: B2 *chậm hơn* một baseline `torch.compile` (không exact), tốc độ co lại khi acceptance thấp, và nhiều hướng cải thiện không mang lại lợi ích. Bằng chứng đều từ một GPU (RTX 4050 Laptop 6 GB), batch 1, greedy; chúng tôi nêu rõ phần lập luận còn mở.

1 Giới thiệu

Giải mã suy đoán tăng tốc suy luận bằng cách đề xuất vài token tương lai rồi kiểm tra chúng song song trong một lần chạy mô hình [1, 2]. Với greedy, đầu ra *về nguyên tắc* bằng đầu ra giải mã tuần tự. Trên thực tế, "bằng nhau" chỉ đúng nếu lần chạy theo khối cho logits và KV giống hệt bước một-token. Các toán tử trong mô hình chọn kernel theo hình dạng đầu vào, nên đầu ra theo khối có thể chỉ gần đúng và quỹ đạo có thể rẽ nhánh; Mục 6.2 cho thấy toán tử nào thực sự gây lệch trong ngăn xếp bitsandbytes 0.50.0 và PyTorch SDPA. Trong giao thức trước của kho mã này, Prompt Lookup Decoding (PLD) [3] trên NF4 [4] chỉ khớp đầu ra không suy đoán ở $83{,}3\%$ chuỗi ($N{=}150$) và $21/60$ ở một phép so sánh backend sau đó.

Mục tiêu. Cho một tham chiếu chính xác (greedy bitsandbytes M=1 không sửa đổi), xây dựng vòng giải mã suy đoán (i) cho đúng chuỗi token tham chiếu, (ii) không dùng mô hình draft và không dùng thêm bộ nhớ GPU đáng kể, (iii) nhanh hơn tham chiếu. Chúng tôi đặt tính đúng đắn lên trước tốc độ: tốc độ chỉ có nghĩa khi (i) đã giữ.

Đóng góp.

1. Một vòng giải mã exact B2 và các thành phần của nó (Mục 3), kèm cách đọc tính đúng đắn theo toán tử và phân loại rõ phần nào theo cấu trúc mã, phần nào chỉ có bằng chứng thực nghiệm (Mục 4).
2. Hai phát hiện: (i) trong ngăn xếp bitsandbytes 0.50.0 và PyTorch, kiểm tra theo khối làm lệch quỹ đạo chủ yếu qua attention và LM head chứ không qua phép chiếu NF4 (Mục 6.2); (ii) mã nguồn CUDA của bitsandbytes đọc theo nghĩa đen cộng tích lũy tuần tự, nhưng bản nhị phân thật khớp thứ tự cộng theo cặp của kernel Triton (Mục 4.1).
3. Bằng chứng nhiều tầng: token và KV bitwise từng bước, quay lui, EOS, ngữ cảnh tới 3071 token, ba kích cỡ mô hình, hai cohort held-out đăng ký trước (Mục 6).
4. Các kết quả bất lợi và âm có ghi chép: baseline biên dịch nhanh hơn, hướng cải thiện không hiệu quả (Mục 7).

Công trình gần nhất chúng tôi biết là LLM-42 [5], dùng vòng decode–verify–rollback để đảm bảo xác định ở mức hệ thống phục vụ trên FP16 nhiều GPU; chúng tôi giải một bài toán khác ở mức số học trong lượng tử hóa 4-bit trên một GPU tiêu dùng.

2 Bài toán và định nghĩa

Cố định trọng số, trạng thái lượng tử hóa, phiên bản thư viện, kiến trúc GPU, cài đặt attention, thủ tục prefill, kiểu và dung lượng cache, quy tắc chọn token khi bằng nhau và quy tắc EOS. Gọi $R(S_t, x_t) = (z_t, S_{t+1})$ là bước chuyển một-token tham chiếu: chạy `model(x)` với bitsandbytes không sửa đổi, x có hình dạng $(1, 1)$. Trạng thái S_t gồm mọi thứ ảnh hưởng tới lần chạy sau (mọi mục KV, độ dài logic, vị trí). *Exact* nghĩa là chuỗi token sinh ra bằng chuỗi do R sinh, và (ở các kiểm tra mạnh hơn) KV cũng bằng từng bit tại mọi bước.

Chúng tôi phân biệt hai tham chiếu. **B1** là greedy với lịch hình dạng cố định (mọi phép toán được đệm về cùng số hàng vật lý); nó làm đổi số học của mô hình nên không bằng greedy gốc. **B2** là greedy bitsandbytes gốc; chỉ B2 là mục tiêu của báo cáo này.

3 Phương pháp

Mỗi chu kỳ gồm: tra n-gram để đề xuất tối đa $k{=}2$ token; chạy mô hình trên $1{+}$#đề xuất hàng; chấp nhận tiền tố khớp dài nhất; quay lui KV phần bị loại. Các thành phần:

Kernel NF4 $M{=}3$ (tùy chọn, chỉ để tăng tốc). `_fused_nf4_m3_kernel` (Triton [6]) xử lý ba hàng activation với ba bộ cộng dồn độc lập, cùng một trọng số giải lượng tử dùng chung. Mỗi lane cộng 32 tích fp16 (`mul.rn.f16x2`), tích lũy trong fp32 theo từng cặp, rồi gộp 32 lane bằng cây dịch $16, 8, 4, 2, 1$. Chạy với `enable_fp_fusion=False`. Mã không có câu lệnh nào đọc dữ liệu của hàng khác nên đầu ra từng hàng chỉ phụ thuộc hàng đó. Kernel này *không cần cho tính exact*: trong bitsandbytes 0.50.0 các phép chiếu NF4 chuẩn đã cho đầu ra từng hàng bằng bước $M{=}1$ (Mục 6.2), và B2 lười không dùng kernel Triton vẫn khớp BNB ở $10/10$ prompt dev (token và KV). Kernel chỉ giúp tăng tốc: $1{,}161\times$ so với $1{,}117\times$ khi dùng phép chiếu chuẩn (10 prompt GSM8K dev).

Phép toán từng hàng. RMSNorm, LM head và attention được gọi riêng cho từng hàng: attention gọi `scaled_dot_product_attention` một lần cho mỗi hàng với K/V cắt đúng $L_t{+}i{+}1$ vị trí, không mặt

Thuật toán 1 Một lượt sinh của B2 (rút gọn)

Đầu vào: prompt, ngân sách N, k=2

prefill bằng BNB gốc; *pending* ← token đầu của greedy
while chưa đủ N token và **chưa** gặp EOS **do**
  $C \leftarrow$ PLD(lịch sử + *pending*, $\leq k$ token)
  if $C = \emptyset$ **then** ▷ quay về bước gốc
    chạy BNB M=1 trên *pending*; *pending* ← argmax
  else
    hoán đổi sang bản vá; chạy decoder trên [*pending*] + C (một lần)
    for $i = 0, 1, \ldots$ **do** ▷ LM head lười
      $p_i \leftarrow$ argmax lm_head(h_i); **nếu** $i < |C|$ **và** $p_i \neq C_i$ **thì** dừng
    end for
    phát *pending* và các C_j được chấp nhận; cắt KV về độ dài đã commit
    pending ← $p_{\text{hàng cuối đã tính}}$
  end if
end while

nạ, `enable_gqa=True`, nên hình dạng giống lời gọi một-token.

Quay lui và quay về M=1. Cache tĩnh chỉ trả về $[0, \text{seq_len})$ nên cắt độ dài logic là đủ để các mục dư không bao giờ được đọc trước khi bị ghi đè. Khi không có đề xuất, vòng chạy đúng bước M=1 gốc (không lịch, không kernel Triton) bằng cách hoán đổi `forward` từng module giữa bản gốc và bản đã vá; lịch được vào một lần cho cả lượt sinh.

LM head lười. Chiếu LM head cho từng hàng chỉ cho tới hàng bị loại đầu tiên: các hàng sau không bao giờ được dùng và không ghi trạng thái, nên bỏ qua chúng không đổi kết quả.

4 Lập luận về tính đúng đắn

Gọi *tính bằng nhau theo hàng* (RE) là: với khối bắt đầu từ trạng thái bằng tham chiếu, hàng i của mọi tensor trung gian bằng từng bit với tensor tương ứng của R áp dụng lần lượt $x_0, \ldots, x_i$, và KV ghi tại vị trí L_t+i cũng bằng. Từ RE, quy nạp theo chu kỳ cho kết quả: (1) logits hàng i bằng tham chiếu; (2) tiền tố được chấp nhận và *pending* bằng chuỗi greedy; (3) cắt độ dài logic giữ lại đúng $[0, L_t{+}j{+}1)$; (4) chu kỳ không đề xuất chính là R. Giả thiết duy nhất không thuộc cấu trúc là RE ở từng toán tử.

4.1 Thứ tự rút gọn của kernel bitsandbytes

Phần này chỉ liên quan tới kernel Triton tùy chọn: nó cho biết khi nào kernel đó thay được kernel bitsandbytes mà không đổi kết quả. Mã nguồn bitsandbytes 0.50.0 (`kgemm_4bit_inference_naive`) cộng tuần tự từng tích fp16 trong mỗi lane, còn kernel Triton của chúng tôi cộng từng cặp tích liền kề trước. Theo số học IEEE hai thứ tự này khác nhau. Chúng tôi mô phỏng cả hai thứ tự trong fp32 trên 28,672,000 đầu ra ngẫu nhiên đuôi nặng (năm lớp, đủ bảy phép chiếu). Kernel bitsandbytes thật và kernel Triton *đều khớp mô phỏng theo cặp ở* 28,672,000/28,672,000 *đầu ra* và đều khác mô phỏng tuần tự ở 398 đầu ra; phép thử đủ nhạy để phân biệt hai thứ tự. Cận trên 95% của tỷ lệ lệch mỗi đầu ra khoảng $1{,}0 \times 10^{-7}$ (quy tắc ba). Chúng tôi chưa giải thích được vì sao bản nhị phân khác mã nguồn đã đọc, nên mọi lần nâng cấp bitsandbytes phải chạy lại phép mô phỏng này.

Bảng 1: Trạng thái lập luận theo toán tử. **[S]**: theo cấu trúc mã; **[E]**: chỉ có bằng chứng thực nghiệm; **[O]**: còn mở.

Toán tử	Cơ sở	Nhãn
Embedding, ghi KV, argmax, LM head	gọi từng hàng hoặc sao chép byte	[S]
RMSNorm	gọi riêng từng hàng, giống tham chiếu	[S]
Attention	một lời gọi SDPA mỗi hàng, cùng hình dạng, bố cục	[S] nếu dispatch chỉ phụ thuộc hình dạng/stride; [O]
Rotary, residual, SiLU·mul	phép toán theo phần tử; chưa kiểm riêng bản vector hóa	[S]/[O]
Phép chiếu NF4 (252)	bitsandbytes chuẩn: 0 lệch trong $9{,}9 \times 10^8$ phần tử khi gọi ba hàng so với từng hàng; kernel Triton tùy chọn có ba bộ cộng dồn độc lập	[E]; [S]+[E] cho kernel Triton

5 Thiết lập thực nghiệm

Môi trường. Qwen2.5-3B-Instruct [7], revision `aa8e7253...`, bitsandbytes 0.50.0 (NF4, lượng tử hóa kép, tính toán fp16), Triton 3.6.0, PyTorch 2.11.0, transformers 5.14.1, một GPU RTX 4050 Laptop 6 GB, batch 1, greedy, k=2, PLD với n-gram ≤ 3, 128 token sinh nếu không ghi khác.

Dữ liệu và giao thức. Tập *dev* (GSM8K, HumanEval, CNN/DM) dùng để phát triển và chọn cấu hình. Hai cohort *held-out* GSM8K, chỉ số test 300–349 và 400–449, được đóng băng theo chỉ số công khai, kiểm tra không trùng ID/prompt với mọi benchmark cục bộ, đăng ký cấu hình trước khi chạy và chỉ dùng *một lần*; chúng không được dùng để chọn cấu hình. Việc này không loại trừ dữ liệu công khai đã lọt vào dữ liệu huấn luyện mô hình.

Chỉ tiêu. (i) Token khớp tham chiếu BNB eager; (ii) băm SHA256 của KV tại mọi tiền tố không-terminal; (iii) tốc độ = thời gian BNB eager / thời gian phương pháp, trung vị trên 2–3 vòng xoay thứ tự, trung bình nhân giữa các prompt kèm khoảng tin cậy 95% bootstrap theo prompt (10^4 lần, seed 0). Thời gian gồm cả prefill.

6 Kết quả

6.1 Tính đúng đắn

6.2 Toán tử nào làm vỡ tính exact

Để biết sai lệch của kiểm tra theo khối đến từ đâu, chúng tôi chạy đường *chuẩn* (không vá) của HF với bitsandbytes 0.50.0 trên các khối ba token teacher-forced, đặt ở trạng thái tuần tự chính xác (20 prompt GSM8K dev, 16 khối mỗi prompt, 960 hàng). Với mỗi toán tử ta ghi đầu vào thật của nó khi chạy khối, rồi gọi lại *cùng toán tử trên cùng đầu vào* từng hàng một và so sánh từng bit (Bảng 3). Chỉ attention và LM head lệch đáng kể; các phép chiếu NF4 chuẩn khớp từng hàng ở mọi phần tử đã kiểm, và RMSNorm gần như khớp. Hệ quả cuối-cuối: $960/960$ hàng logits không bằng từng bit, $4/960$ hàng ($0{,}42\%$) đổi argmax (độ chênh top-1/top-2 của chúng $\leq 0{,}125$, một hàng là hòa tuyệt đối); trung vị của độ lệch logits lớn nhất là $0{,}22$ và giá trị lớn nhất là $1{,}36$. Vì vậy trong ngăn xếp này tính exact bị phá bởi attention và LM head

Bảng 2: Bằng chứng tính đúng đắn của B2 (so với greedy bitsandbytes gốc).

Kiểm tra	Quy mô	Kết quả
Held-out GSM8K 300–349 (token)	50 prompt	$50/50$
Held-out GSM8K 400–449 (token và KV từng bước)	50 prompt	$50/50$ và $50/50$
Dev GSM8K, HumanEval (token)	60 prompt	$60/60$
Ngữ cảnh 512–3071 token (token và KV)	12 prompt	$12/12$
Kết thúc bằng EOS, sinh tới 384 token (token và KV)	18 prompt	$18/18$
Quay lui/loại tại các vị trí $0, 1$, nhiều chu kỳ (3 prompt trên 3B, 1 prompt mỗi mô hình 0.5B và 1.5B)	5 kịch bản	đạt
Qwen2.5-0.5B và 1.5B (token và KV)	10 prompt mỗi mô hình	$10/10$
Phép chiếu NF4 trên activation thật	252 module, 9,29M phần tử	bằng từng bit

Bảng 3: So sánh từng bit giữa toán tử gọi ba hàng và gọi từng hàng trên cùng đầu vào (đường chuẩn, 960 hàng).

Toán tử	Số phần tử	Lệch từng bit	Tỷ lệ
Phép chiếu NF4 (7 loại)	990 904 320	0	0
RMSNorm trong lớp	141 557 760	87	$6{,}1 \times 10^{-7}$
RMSNorm cuối	1 966 080	0	0
LM head	145 858 560	281 534	$1{,}9 \times 10^{-3}$
Attention (SDPA)	70 778 880	13 196 548	0,186

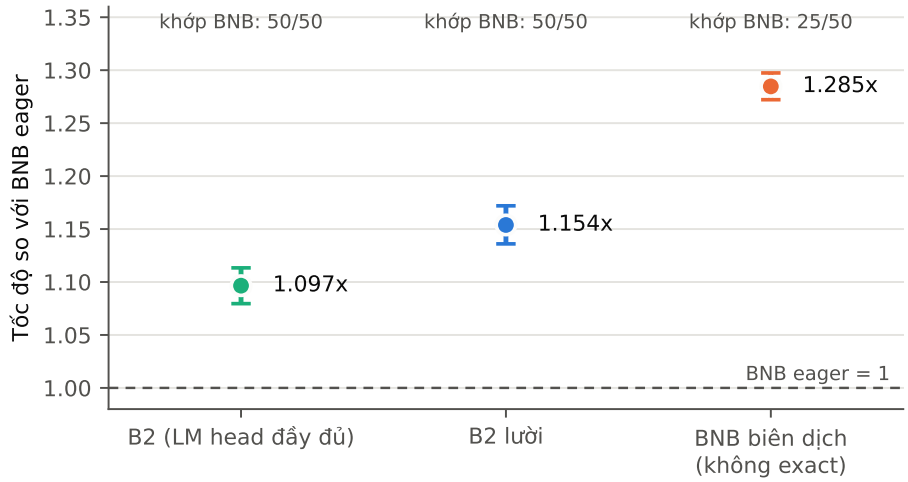
chứ không phải phép chiếu 4-bit, và B2 xử lý chính các toán tử đó bằng lời gọi từng hàng. Kết quả chỉ cho một phiên bản bitsandbytes, một GPU và $M{=}3$: bitsandbytes 0.50.0 chọn kernel NF4 cho M nhỏ theo ngưỡng riêng, và phiên bản khác có thể khác.

6.3 Tốc độ và so sánh với B1

Trên held-out 300–349 (phiên bản B2 với LM head đầy đủ), B2 đạt 1,117 [1,100, 1,135]$\times$ so với BNB eager, khớp $50/50$ prompt; B1 (kernel Triton ba hàng, lịch hình dạng cố định) đạt 1,108 [1,087, 1,130]$\times$ nhưng chỉ khớp BNB gốc ở $24/50$ prompt. Tỷ số B2/B1 là 1,008 [0,995, 1,022]: hai bên không phân biệt được về tốc độ, khác biệt nằm ở tính exact. Trên held-out 400–449 với B2 lười (Hình 1): B2 lười đạt 1,154 [1,136, 1,172]$\times$ so với 1,097 [1,080, 1,113]$\times$ của B2 LM head đầy đủ (lười/đầy đủ $= 1{,}052[1{,}044,\ 1{,}060]$), không prompt nào dưới BNB. Trên dev, B2 lười đạt 1,166 [1,145, 1,188]$\times$ (GSM8K) và 1,249 [1,208, 1,296]$\times$ (HumanEval), mỗi cohort 30 prompt.

6.4 So sánh với baseline biên dịch

Một vòng decode BNB với `StaticCache` và `torch.compile` (mode `reduce-overhead`) nhanh hơn BNB eager 1,285 [1,272, 1,297]$\times$ trên held-out 400–449, đo cạnh nhau cùng lượt chạy. Tuy nhiên token của nó chỉ khớp eager ở $25/50$ prompt, tức không phải tham chiếu bitwise. B2 lười đạt $0{,}907\times$ [0,893, 0,920] tốc độ của nó và chậm hơn ở $49/50$ prompt (trên dev: $0{,}82\times$, 12 prompt). Vì vậy luận điểm đúng của B2 là "exact so với BNB eager, nhanh hơn eager khoảng $1{,}1$–$1{,}25\times$", không phải "nhanh hơn baseline mạnh". Chưa thử kết hợp compile với giải mã suy đoán; nó sẽ đổi số học và đòi hỏi định nghĩa lại tham chiếu.



Hình 1: Tốc độ so với BNB eager trên held-out GSM8K 400–449 (50 prompt; điểm là trung bình nhân, thanh là khoảng tin cậy 95%). Dòng trên cùng ghi số prompt có token khớp BNB eager. BNB biên dịch nhanh hơn nhưng chỉ khớp $25/50$.

6.5 Ngữ cảnh dài và các mô hình khác

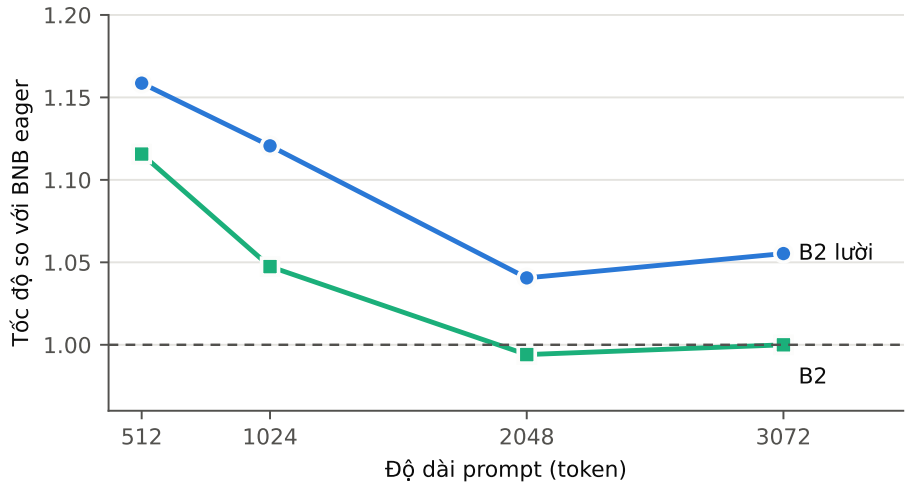
Token và KV khớp ở mọi độ dài tới 3071 token, nhưng tốc độ giảm (Hình 2): B2 lười còn $1{,}04$–$1{,}06\times$ ở 2048–3072 token. Chi phí một forward gần như không đổi theo độ dài (wall: BNB $M{=}1$ $23{,}2 \rightarrow 23{,}5$ ms), nên nguyên nhân đã đo là acceptance thấp trên tác vụ tóm tắt ($0{,}24$–$0{,}29$ ở 3072 so với $0{,}32$–$0{,}64$ ở 512) và prefill chung ($1{,}1$ s ở 3072 token) làm loãng tỷ lệ; chưa có prompt dài mà acceptance cao để tách hoàn toàn hai yếu tố. Trên Qwen2.5-0.5B và 1.5B (10 prompt dev) B2 lười đạt $1{,}243\times$ và $1{,}163\times$, token và KV khớp $10/10$.

6.6 Bộ nhớ

Ở cùng dung lượng cache tĩnh, đỉnh bộ nhớ cấp phát của B2 chỉ lớn hơn BNB eager $0{,}07$ MiB ($2007{,}90$ so với $2007{,}97$ MiB ở prompt 75 token; $2994{,}61$ so với $2994{,}68$ MiB ở 3071 token); đỉnh dự trữ và bộ nhớ thiết bị dùng giống hệt nhau, và không có mô hình draft. Mỗi phương pháp chạy trong một tiến trình riêng, đo ở lượt thứ hai.

6.7 Phân tích chi phí

Một forward khối $M{=}3$ tốn $37{,}1$ ms, gấp $1{,}65$ lần một bước $M{=}1$ ($22{,}5$ ms), nên block chỉ có lợi khi sinh trên $1{,}65$ token mỗi chu kỳ (acceptance khoảng 33% với hai đề xuất); trung bình $1{,}86$ token mỗi chu kỳ khối và 73% chu kỳ phải quay về bước gốc, khớp với mức tăng tốc khoảng 12–15%. LM head bf16 tốn khoảng $3{,}3$ ms mỗi hàng và là phần lớn của chênh lệch GPU, nên chiếu lười có ích. Cả hai loại forward bị giới hạn bởi CPU khởi chạy kernel (84% thời gian một bước $M{=}1$ là enqueue CPU), điều này cũng giải thích vì sao baseline biên dịch (giảm overhead CPU) nhanh hơn.



Hình 2: Tốc độ so với BNB eager theo độ dài prompt (3 prompt dev mỗi độ dài, tóm tắt nhiều bài CNN/DM ghép lại).

6.8 Thăm dò: chụp CUDA graph (ước lượng, chưa cài đặt)

Chúng tôi chụp thô một bước decode $M{=}1$ của mô hình bitsandbytes eager với `StaticCache` bằng CUDA graph (không `torch.compile`, cùng các kernel): chụp được, logits bằng từng bit ở $64/64$ bước, và chi phí giảm từ $22{,}5$ xuống $17{,}3$ ms mỗi bước ($1{,}30\times$). Khối chuẩn ba hàng (không exact) tốn $27{,}4$ ms ở chế độ eager và $23{,}9$ ms trong graph. Ghép với mô phỏng đã hiệu chỉnh, một vòng B2 chạy phần thân decoder trong graph (các phép từng hàng nằm trong graph nên không còn tốn CPU) với LM head lười chạy ngoài graph cho tốc độ ước lượng $1{,}36$–$1{,}48\times$ so với eager, nhưng chỉ $1{,}04$–$1{,}14\times$ so với một baseline graph exact. Đây là dự đoán từ mô hình chi phí (chi phí decoder ba hàng trong graph $21{,}7$–$24{,}0$ ms ước từ số đo kernel), chưa được cài đặt, nên không phải kết quả đã đo của B2.

7 Các kết quả bất lợi và hướng không hiệu quả

- **Chọn n-gram.** Quét độ dài n-gram 2–5 chênh dưới 2% và nằm trong nhiễu đo. Bảy chính sách chọn draft (lọc đề xuất yếu, bỏ phiếu giữa các lần khớp, ngưỡng lặp) đều khớp BNB; lọc đề xuất yếu không nhanh hơn dù giảm gần một nửa số khối, và bỏ phiếu so với chính sách gốc cho tỷ số $1{,}002$ $[0{,}998, 1{,}007]$ trên 80 prompt dev chưa dùng để chọn. Giữ chính sách gốc.
- **Giảm chi phí CPU.** Gọi thẳng kernel Triton đã biên dịch cho kết quả bằng nhau từng bit nhưng không nhanh hơn ($7{,}22$ so với $7{,}59$ ms cho 252 phép chiếu); các điểm nóng còn lại là chuỗi phép PyTorch nhỏ ở RMSNorm và attention từng hàng, không thể giảm mà không đổi tập kernel.
- **Khối lớn hơn.** Mô phỏng (chạy PLD trên chuỗi tham chiếu đã ghi, chi phí mỗi hàng hiệu chỉnh khớp $K{=}2$ đo được) dự đoán $K{=}3, 4$ thay đổi từ -1% đến $+2{,}6\%$ so với $K{=}2$ (tới $+4{,}1\%$ nếu chi phí mỗi hàng giảm một nửa); không đáng viết kernel mới. Đây là dự đoán từ mô hình tuyến tính, chưa kiểm bằng thực nghiệm.
- **Quay về bước gốc.** Quay về $M{=}1$ khi không có đề xuất đưa tốc độ từ $0{,}91$–$0{,}97\times$ lên $1{,}06$–$1{,}15\times$ trên các cohort dev nhỏ; đây là thay đổi đóng góp lớn nhất.

8 Giới hạn và mối đe dọa tính hợp lệ

- **Phạm vi.** Một GPU và bộ công cụ, chủ yếu một họ mô hình (Qwen2), một backend lượng tử hóa (bitsandbytes), batch 1, greedy, $k{=}2$. Kết quả không áp dụng được cho lấy mẫu, batch lớn hay các lược đồ lượng tử hóa khác.
- **Chứng minh.** Tính exact dựa trên bằng chứng hữu hạn. Còn mở: bằng kernel bitsandbytes trên mọi đầu vào, độc lập của dispatch PyTorch với số hàng, độ ổn định qua phiên bản Triton/ptxas/CUDA và GPU, kiểm ở mức PTX.
- **Chọn cấu hình.** Cấu hình được chọn trên dev; hai cohort held-out không dùng để chọn, nhưng cohort 400–449 được dùng để đánh giá riêng biến thể B2 lười đã cố định trước đó, và cohort 300–349 chỉ đánh giá biến thể LM head đầy đủ.
- **Khoảng tin cậy.** Chỉ phản ánh biến thiên giữa các prompt đã chọn, không phải toàn bộ workload. Tốc độ tuyệt đối phụ thuộc cohort ($1{,}23\times$ ở HumanEval 0–19 nhưng $1{,}31\times$ ở 20–49).
- **Baseline.** Tốc độ chỉ so với BNB eager; baseline biên dịch nhanh hơn nhưng không exact.

9 Kết luận

Với Qwen2.5-3B NF4 trên một GPU tiêu dùng, B2 tái tạo đúng từng token và KV của greedy bitsandbytes, không dùng thêm bộ nhớ đáng kể và nhanh hơn eager khoảng 12–15% trên hai cohort held-out; mức tăng khiêm tốn và bị giới hạn bởi chi phí khởi chạy CPU và acceptance của PLD. Giá trị chính nằm ở tính exact và quy trình bằng chứng chứ không ở tốc độ. Hướng tiếp theo gồm kiểm ở mức PTX và trên GPU/phiên bản Triton khác, mở rộng sang backend lượng tử hóa và họ mô hình khác, và chứng minh hợp đồng toán tử.

Mã nguồn và bằng chứng. Mã, bằng chứng thô, manifest ghim SHA256, kịch bản tái lập, lập luận theo toán tử và ghi chép trạng thái nghiên cứu đều nằm trong kho mã của dự án (thư mục `experiments` và các tệp ở thư mục gốc); số liệu trong báo cáo được lấy trực tiếp từ các tệp JSON bằng chứng đó.

Tài liệu

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[2] Charlie Chen, Sebastian Borgeaud, et al. Accelerating large language model decoding with speculative sampling. In *arXiv preprint arXiv:2302.01318*, 2023.

[3] Apurv Saxena. Prompt lookup decoding. *GitHub repository*, 2024.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[5] Raja Gond, Aditya K Kamath, Ramachandran Ramjee, and Ashish Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2026.

[6] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.

[7] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.